

Finite-Pulse Bragg Transfer-Function Calculator

Theory, numerical methods and user manual

MIGA programme

Version 1.0 • 1 September 2026

Abstract

This manual documents a local Python application for calculating the sensitivity function and laser-phase transfer function of a finite-pulse, three-pulse Bragg atom interferometer. The program supports resonant square pulses and Gaussian optical-intensity pulses, allows independent durations for the $\pi/2$ and π interactions, and applies the MIGA convention in which the Bragg diffraction order multiplies the interferometer phase response. Square-pulse transfer functions are evaluated analytically using a numerically stable sinc representation. Gaussian-pulse transfer functions are evaluated by Gauss–Legendre quadrature after normalizing each pulse to its required area. This document explains the physical model, timing conventions, formulas, numerical implementation, graphical interface, power-spectral-density interpretation, validation checks and known limitations. It is written as a self-contained Overleaf-compatible manual rather than as a journal submission.

i What the software calculates

The displayed transfer function maps the local phase difference of the two counter-propagating Bragg fields to the atom-interferometer phase. It does not directly map mirror displacement, acceleration or the frequency noise of an individual laser unless the corresponding optical phase conversion is supplied separately.

Contents

1	Purpose and scope	3
2	Quick start	3
2.1	Software requirements	3
2.2	Minimal operating procedure	3
3	Parameters and timing conventions	4
4	Physical model	4
4.1	Sensitivity-function relation and Bragg order	4
4.2	Pulse areas	5
5	Square-pulse calculation	6
5.1	Effective Rabi frequencies	6
5.2	Sensitivity function	6
5.3	Stable analytic Fourier integral	6

6	Gaussian-pulse calculation	7
6.1	Optical-intensity FWHM	7
6.2	Finite numerical window and accumulated area	7
6.3	Gauss–Legendre quadrature	8
7	Transfer functions and spectral densities	8
7.1	Magnitude, phase and power transfer	8
7.2	Sinusoidal frequency modulation	8
8	Interpreting the plots	9
8.1	Sensitivity function	9
8.2	Frequency fringes and the finite-pulse envelope	9
8.3	Sampling and apparent high-frequency disorder	10
9	Detailed UI reference	10
9.1	Pulse-sequence controls	10
9.2	Frequency controls	10
9.3	Displayed status information	11
9.4	Saving and inspecting figures	11
10	Worked examples	11
10.1	Default first-order calculation	11
10.2	Changing Bragg order	11
10.3	Evaluating a modulation line	11
11	Validation and reproducibility	12
12	Troubleshooting	12
13	Known limitations and extension path	13
14	Code organization	13
15	Using this manual on Overleaf	13

1 Purpose and scope

The application `bragg_transfer_function_ui.py` evaluates a three-pulse Mach–Zehnder sequence with pulse areas $\pi/2-\pi-\pi/2$. It follows the sensitivity-function description used for MIGA[1] and the time-domain atom-interferometer formalism described by Cheinet *et al.*[2]. The software is intended for rapid exploration of finite-pulse filtering, comparison of square and Gaussian envelopes, and conversion of an input laser-phase spectrum into an output atom-phase spectrum.

The model assumes resonant driving, an effective two-level system, non-overlapping pulses and ideal pulse areas. It does not propagate a full momentum-state Hamiltonian. Consequently, a higher Bragg order n changes the phase gain but not the normalized shape of the sensitivity function. This approximation is useful when the effective pulse widths are known experimentally, but it does not predict diffraction efficiency, parasitic momentum populations, velocity selectivity, detuning, AC Stark shifts or cavity dynamics.

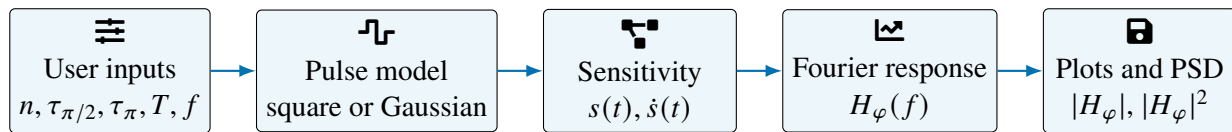


Figure 1. Calculation workflow. The application converts the pulse parameters into a sensitivity function, Fourier-transforms its derivative and displays the magnitude of the phase transfer function. The square branch is analytic, whereas the Gaussian branch uses numerical quadrature.

2 Quick start

2.1 Software requirements

The program requires Python 3.10 or later, NumPy, Matplotlib and Tkinter. Tkinter is included with most standard Python distributions. No browser, web server or SciPy installation is required. From PowerShell, launch the application from its directory with

```
python "bragg_transfer_function_ui.py"
```

The default parameter set is $n = 1$, $\tau_{\pi/2} = 25 \mu\text{s}$, $\tau_{\pi} = 50 \mu\text{s}$ and $T = 250 \text{ ms}$. The default frequency grid extends from 0.1 Hz to 100 kHz.

2.2 Minimal operating procedure

1. Select Square or Gaussian in the Pulse shape menu.
2. Enter a positive integer Bragg order n .
3. Enter the $\pi/2$ and π pulse widths in microseconds. In Gaussian mode these values are optical-intensity FWHM values.
4. Enter the interrogation time T in milliseconds. Its precise timing definition depends on pulse shape and is stated in [section 3](#).
5. Choose a positive minimum frequency, a larger maximum frequency and the number of logarithmically spaced frequency points.
6. Press Calculate or the Return key. Use the Matplotlib toolbar to zoom, pan or inspect coordinates.
7. Press Save figure... to export the three displayed panels as PNG, PDF or SVG.

The first panel is a compressed timing schematic. Its pulse widths and T annotations carry the physical values, even though the long free-evolution gaps are shortened visually. The toolbar coordinate readout reports relative physical time in milliseconds. The second panel displays $s(t)$ on the full sequence time axis, and the third panel displays $|H_{\varphi}(f)|$.

3 Parameters and timing conventions

Table 1. User inputs and their interpretation.

Input	Symbol	UI unit	Meaning
Bragg order	n	integer	Phase-response multiplier in the MIGA convention.
$\pi/2$ width	$\tau_{\pi/2}$	μs	Square duration or Gaussian optical-intensity FWHM. The first and third pulses share this value.
π width	τ_{π}	μs	Square duration or Gaussian optical-intensity FWHM. It may differ from $2\tau_{\pi/2}$.
Interrogation time	T	ms	Edge-to-edge free time in square mode; centre-to-centre pulse separation in Gaussian mode.
Frequency limits	$f_{\min}, f_{\max}$	Hz	Positive endpoints of the logarithmic plotting grid.
Frequency points	N_f	integer	Number of plotted frequencies, from 100 to 100,000.

For square pulses, the first pulse occupies $0 \leq t < \tau_{\pi/2}$, the second starts after a free interval T , and the third starts after a second free interval T . Defining

$$t_2 = \tau_{\pi/2} + T, \quad (1)$$

$$t_3 = \tau_{\pi/2} + T + \tau_{\pi} + T, \quad (2)$$

the pulse intervals are $[0, \tau_{\pi/2}]$, $[t_2, t_2 + \tau_{\pi}]$ and $[t_3, t_3 + \tau_{\pi/2}]$. The total sequence duration is

$$T_{\text{seq,sq}} = 2T + 2\tau_{\pi/2} + \tau_{\pi}. \quad (3)$$

For Gaussian pulses, mathematical tails have no unique edge. The program therefore defines the pulse centres as $t_{c,1}$, $t_{c,1} + T$ and $t_{c,1} + 2T$. Each pulse is represented numerically over $\pm 6\sigma$ and normalized over that interval. A calculation is rejected if these numerical windows overlap, because the separated-pulse sensitivity formulas would then no longer represent the coupled dynamics.

Do not compare the two modes using an ambiguous T

Square-mode T is an edge gap, whereas Gaussian-mode T is a centre spacing. To align pulse centres for a controlled comparison, convert the square value using $T_{\text{centre}} = T_{\text{edge}} + (\tau_{\pi/2} + \tau_{\pi})/2$. The same conversion applies between the second and third pulses because their adjacent widths are again τ_{π} and $\tau_{\pi/2}$.

4 Physical model

4.1 Sensitivity-function relation and Bragg order

Let $\Delta\varphi(t)$ denote the local phase difference of the two counter-propagating interrogation fields. In the MIGA convention, the atom-interferometer phase response is[1]

$$\Delta\Phi_{\text{AI}}(t) = n \Delta\varphi(t) * \frac{ds(t)}{dt}, \quad (4)$$

where $*$ is convolution and $s(t)$ is the normalized sensitivity function. With the Fourier convention

$$X(\omega) = \int_{-\infty}^{\infty} x(t) e^{-i\omega t} dt, \quad (5)$$

the phase transfer function is

$$H_\varphi(\omega) = n \int_{-\infty}^{\infty} \dot{s}(t) e^{-i\omega t} dt = n i \omega G(\omega), \quad (6)$$

where $G(\omega)$ is the Fourier transform of $s(t)$. The application plots $|H_\varphi|$. Because n appears outside $s(t)$, increasing the Bragg order scales $|H_\varphi|$ by n and $|H_\varphi|^2$ by n^2 without changing the plotted shape of $s(t)$.

4.2 Pulse areas

For resonant effective two-level evolution, the accumulated pulse area is

$$\theta(t) = \int^t \Omega(t') dt'. \quad (7)$$

The complete areas are $\Theta_1 = \pi/2$, $\Theta_2 = \pi$ and $\Theta_3 = \pi/2$. This area condition, rather than a fixed relation between the pulse widths, ensures the required rotations. Consequently, the program permits $\tau_\pi \neq 2\tau_{\pi/2}$ and adjusts the effective Rabi rate accordingly.

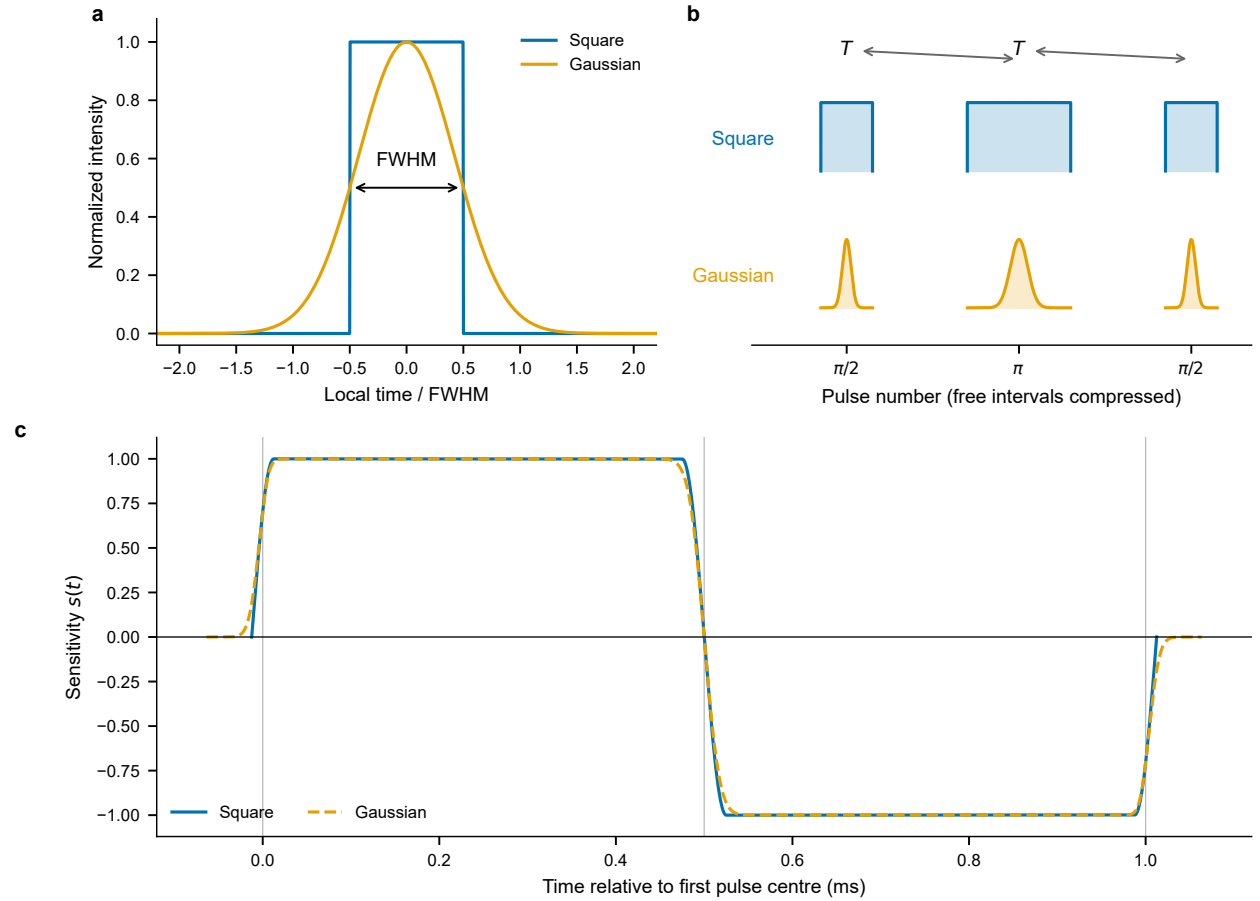


Figure 2. Pulse envelopes and sensitivity functions. **a**, A square envelope and a Gaussian optical-intensity envelope with the same FWHM. **b**, Schematic $\pi/2$ – π – $\pi/2$ sequences; free intervals are compressed. **c**, Sensitivity functions for illustrative centre separations of 0.5 ms, with $\tau_{\pi/2} = 25 \mu\text{s}$ and $\tau_\pi = 50 \mu\text{s}$. The square edge gap is converted so that the pulse centres coincide. The two shapes agree on the plateaus and differ only during the finite interactions.

5 Square-pulse calculation

5.1 Effective Rabi frequencies

For a resonant square pulse, $\theta = \Omega\tau$. Independent $\pi/2$ and π widths therefore imply

$$\boxed{\Omega_{\pi/2} = \frac{\pi}{2\tau_{\pi/2}}, \quad \Omega_{\pi} = \frac{\pi}{\tau_{\pi}}.} \quad (8)$$

The UI reports $\Omega/(2\pi)$ in hertz. If $\tau_{\pi} = 2\tau_{\pi/2}$, both pulses have the same peak effective Rabi rate.

5.2 Sensitivity function

With the timing variables defined in [section 3](#), the square-pulse sensitivity function is

$$s_{\text{sq}}(t) = \begin{cases} 0, & t < 0, \\ \sin(\Omega_{\pi/2}t), & 0 \leq t < \tau_{\pi/2}, \\ 1, & \tau_{\pi/2} \leq t < t_2, \\ \cos[\Omega_{\pi}(t - t_2)], & t_2 \leq t < t_2 + \tau_{\pi}, \\ -1, & t_2 + \tau_{\pi} \leq t < t_3, \\ -\cos[\Omega_{\pi/2}(t - t_3)], & t_3 \leq t < t_3 + \tau_{\pi/2}, \\ 0, & t \geq t_3 + \tau_{\pi/2}. \end{cases} \quad (9)$$

Its derivative is non-zero only during the three pulses:

$$\dot{s}_{\text{sq}}(t) = \begin{cases} \Omega_{\pi/2} \cos(\Omega_{\pi/2}t), & 0 < t < \tau_{\pi/2}, \\ -\Omega_{\pi} \sin[\Omega_{\pi}(t - t_2)], & t_2 < t < t_2 + \tau_{\pi}, \\ \Omega_{\pi/2} \sin[\Omega_{\pi/2}(t - t_3)], & t_3 < t < t_3 + \tau_{\pi/2}, \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

The integrals of these three pieces are +1, -2 and +1. Their sum is zero, which enforces $H_{\varphi}(0) = 0$.

5.3 Stable analytic Fourier integral

Direct formulas containing denominators $\omega \pm \Omega$ are formally singular at $\omega = \mp\Omega$, even though the physical limits are finite. The code avoids these removable singularities by defining

$$E(q, d) = d e^{iqd/2} \text{sinc}\left(\frac{qd}{2}\right), \quad \text{sinc}(x) = \frac{\sin x}{x}. \quad (11)$$

The required local integrals are

$$C(\omega, \Omega, d) = \frac{E(\Omega - \omega, d) + E(-\Omega - \omega, d)}{2}, \quad (12)$$

$$S(\omega, \Omega, d) = \frac{E(\Omega - \omega, d) - E(-\Omega - \omega, d)}{2i}. \quad (13)$$

The complete square-pulse result is

$$\boxed{\frac{H_{\varphi}^{\text{sq}}(\omega)}{n} = \Omega_{\pi/2} C(\omega, \Omega_{\pi/2}, \tau_{\pi/2}) - \Omega_{\pi} e^{-i\omega t_2} S(\omega, \Omega_{\pi}, \tau_{\pi}) + \Omega_{\pi/2} e^{-i\omega t_3} S(\omega, \Omega_{\pi/2}, \tau_{\pi/2})} \quad (14)$$

NumPy's `sinc` uses the normalized definition $\sin(\pi x)/(\pi x)$, so the code passes $qd/(2\pi)$ to obtain the mathematical definition used above.

6 Gaussian-pulse calculation

6.1 Optical-intensity FWHM

The Gaussian input width is defined as the full width at half maximum of the optical intensity:

$$I_j(t) = I_{0,j} \exp \left[-4 \ln 2 \left(\frac{t - t_{c,j}}{\tau_j} \right)^2 \right], \quad (15)$$

where τ_j is the entered FWHM. The corresponding standard deviation is

$$\sigma_j = \frac{\tau_j}{2\sqrt{2 \ln 2}}. \quad (16)$$

The software assumes that the two counter-propagating Bragg fields have matched intensity envelopes. Their two-photon effective Rabi rate is then proportional to the product of their field amplitudes and therefore proportional to the common optical intensity envelope. Under this assumption,

$$\Omega_j(t) = \Omega_{0,j} \exp \left[-\frac{(t - t_{c,j})^2}{2\sigma_j^2} \right] \quad (17)$$

has the same FWHM as [equation \(15\)](#). For an ideal infinite Gaussian normalized to total area Θ_j ,

$$\Omega_{0,j} = \frac{\Theta_j}{\sigma_j \sqrt{2\pi}} = \frac{2\sqrt{\ln 2}}{\sqrt{\pi}} \frac{\Theta_j}{\tau_j}. \quad (18)$$

⚠ When optical-intensity FWHM is not the Rabi FWHM

If only one Bragg beam is intensity-modulated while the other remains constant, the effective two-photon Rabi rate is proportional to the modulated field amplitude, $\sqrt{I(t)}$, rather than to $I(t)$. In that configuration the Rabi envelope has a different FWHM, and the current Gaussian conversion must be modified.

6.2 Finite numerical window and accumulated area

The program truncates each Gaussian at $L_j = 6\sigma_j$ and renormalizes it over $[-L_j, L_j]$. Writing $e_6 = \text{erf}(6/\sqrt{2})$, the implemented envelope and accumulated local area are

$$\Omega_j(u) = \frac{\Theta_j}{\sigma_j \sqrt{2\pi} e_6} \exp \left(-\frac{u^2}{2\sigma_j^2} \right), \quad (19)$$

$$\theta_j(u) = \Theta_j \frac{\text{erf}[u/(\sqrt{2}\sigma_j)] + e_6}{2e_6}, \quad -6\sigma_j \leq u \leq 6\sigma_j. \quad (20)$$

This finite-window normalization makes $\theta_j(-6\sigma_j) = 0$ and $\theta_j(6\sigma_j) = \Theta_j$ to numerical precision.

For separated pulses, the pulse-local sensitivity functions have the same rotation structure as the square case:

$$s_1(t) = \sin \theta_1(t), \quad \dot{s}_1(t) = \Omega_1(t) \cos \theta_1(t), \quad (21)$$

$$s_2(t) = \cos \theta_2(t), \quad \dot{s}_2(t) = -\Omega_2(t) \sin \theta_2(t), \quad (22)$$

$$s_3(t) = -\cos \theta_3(t), \quad \dot{s}_3(t) = \Omega_3(t) \sin \theta_3(t). \quad (23)$$

The free-evolution plateaus remain +1 and -1. The Gaussian transfer function is then evaluated from [equation \(6\)](#).

6.3 Gauss–Legendre quadrature

Each pulse integral is mapped to the standard Gauss–Legendre interval. The code selects the number of nodes according to

$$N_q = \max[256, \lceil 20f_{\max}\Delta t_j \rceil + 64], \quad (24)$$

where $\Delta t_j = 12\sigma_j$ is the numerical pulse window. A hard limit of 4096 nodes prevents accidental calculations with unreasonable memory or runtime. Frequencies are processed in chunks of 2048, so a large frequency grid does not require one enormous complex-valued phase matrix.

The Gaussian calculation is numerical, but it is not a coarse time-grid FFT. It evaluates the requested frequencies directly and uses high-order quadrature inside each smooth pulse. This distinction avoids dependence on an arbitrary global time step and accurately represents very long free intervals through complex phase factors.

7 Transfer functions and spectral densities

7.1 Magnitude, phase and power transfer

If $S_{\Delta\varphi}(f)$ is the one-sided power spectral density (PSD) of the local differential optical phase, the atom-phase PSD is

$$S_{\Phi_{\text{AI}}}(f) = |H_{\varphi}(f)|^2 S_{\Delta\varphi}(f). \quad (25)$$

For amplitude spectral density (ASD), the corresponding relation is

$$\sqrt{S_{\Phi_{\text{AI}}}(f)} = |H_{\varphi}(f)| \sqrt{S_{\Delta\varphi}(f)}. \quad (26)$$

Thus ASD is multiplied by $|H_{\varphi}|$, whereas PSD is multiplied by $|H_{\varphi}|^2$. The present UI plots the former quantity. The complex transfer phase is calculated internally but is not displayed because its value depends on the chosen time origin; the magnitude is invariant under a global time translation.

7.2 Sinusoidal frequency modulation

For a deterministic laser-frequency modulation

$$\delta\nu(t) = \Delta\nu \cos(2\pi f_m t + \phi_0), \quad (27)$$

the corresponding optical phase modulation follows from $\dot{\varphi} = 2\pi\delta\nu$:

$$\delta\varphi(t) = \frac{\Delta\nu}{f_m} \sin(2\pi f_m t + \phi_0). \quad (28)$$

The atom-phase peak amplitude is therefore

$$\Delta\Phi_{\text{AI,pk}} = |H_{\varphi}(f_m)| \frac{\Delta\nu}{f_m}. \quad (29)$$

In a one-sided ideal PSD, the deterministic line has integrated power

$$\int_{\text{line}} S_{\Phi_{\text{AI}}}(f) df = \frac{1}{2} |H_{\varphi}(f_m)|^2 \left(\frac{\Delta\nu}{f_m} \right)^2. \quad (30)$$

For a finite record, the line is broadened by the observation time and window function. Its integrated area, rather than its tallest PSD bin, is the stable quantity.

⚠ Common-mode frequency modulation

The conversion above assumes that the modulation produces the differential phase $\Delta\varphi(t)$ seen by the atoms. A modulation applied to a common laser may cancel between the two Bragg fields or acquire an additional propagation-delay transfer function. Optical path difference, AOM architecture and which beam is modulated must be included before applying [equation \(25\)](#).

8 Interpreting the plots

8.1 Sensitivity function

The sensitivity function begins at zero, rises continuously to +1 during the first beam splitter, changes from +1 to -1 during the mirror pulse, and returns from -1 to zero during the final beam splitter. When T is many orders of magnitude longer than the pulse widths, the full-sequence plot visually resembles an ideal step function. The pulse-shape differences remain present but are confined to very narrow regions. The first UI panel therefore uses compressed gaps to make the square or Gaussian envelopes and their FWHM values visible.

8.2 Frequency fringes and the finite-pulse envelope

In the instantaneous-pulse limit,

$$H_\varphi(\omega) = n(1 - e^{-i\omega T})^2, \quad |H_\varphi| = 4n \sin^2\left(\frac{\omega T}{2}\right). \quad (31)$$

The fringe period in ordinary frequency is approximately

$$\Delta f_{\text{fringe}} = \frac{1}{T}. \quad (32)$$

For $T = 250$ ms, the period is 4 Hz. This fine structure is produced by coherent addition of the three pulse contributions. A sinusoidal phase perturbation at a transfer-function zero is rejected, whereas one at a neighbouring maximum is transmitted strongly.

The pulse duration sets a separate, much higher frequency scale. When $f \ll 1/\tau$, a perturbation changes little during a pulse and square and Gaussian responses are nearly identical. Their difference becomes clear near the effective Rabi frequency and above. Abrupt square edges produce slowly decaying Fourier side lobes, while the smooth Gaussian envelope suppresses high-frequency response more strongly.

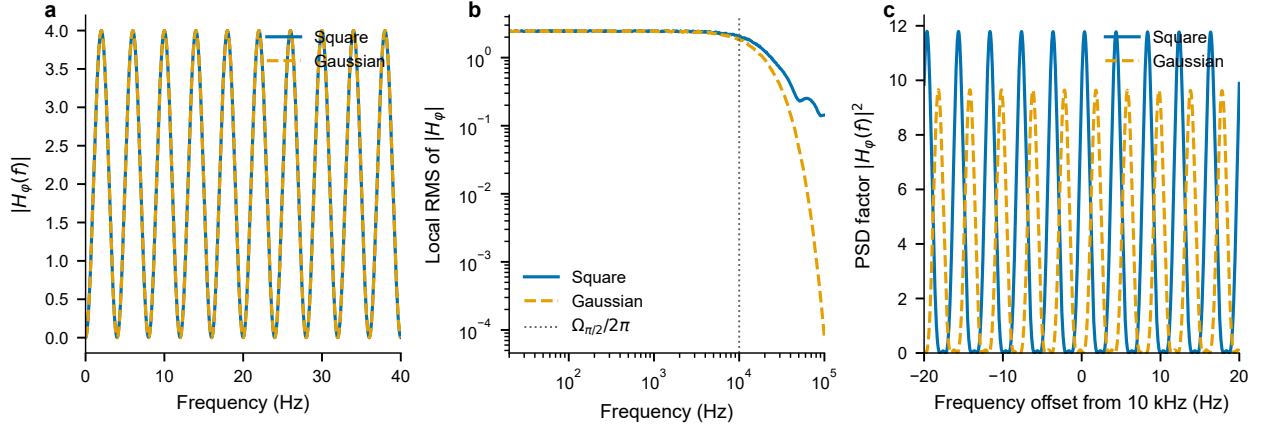


Figure 3. Frequency response for $n = 1$, $\tau_{\pi/2} = 25 \mu\text{s}$, $\tau_\pi = 50 \mu\text{s}$ and $T = 250 \text{ ms}$. **a**, Resolved low-frequency magnitude; the 4 Hz fringe period is set by $1/T$. **b**, Local root-mean-square magnitude, obtained by averaging $|H_\phi|^2$ over one fringe period before taking the square root. The pulse shape controls the high-frequency envelope, and the dotted line marks $\Omega_{\pi/2}/(2\pi)$ for the square pulse. **c**, Resolved PSD transfer factor in a 40 Hz interval around 10 kHz. At a specific coherent modulation frequency, the exact fringe value matters; for broadband noise, a local average is often more informative.

8.3 Sampling and apparent high-frequency disorder

The UI uses a logarithmic frequency grid. At high frequency, adjacent logarithmic points may be separated by much more than $1/T$. Connecting those undersampled points can make a deterministic fringe pattern appear irregular or noisy. This is a plotting alias, not stochastic physics and not necessarily a quadrature error.

To resolve the fringes in a narrow interval, choose a linear-equivalent spacing satisfying approximately

$$\Delta f \lesssim \frac{1}{10T}. \quad (33)$$

The current UI always constructs a logarithmic grid, so the practical procedure is to narrow the frequency range around the region of interest and increase the number of points. For broadband PSD calculations over a very wide band, use a local average of $|H_\phi|^2$ or integrate the PSD directly rather than trying to draw every fringe.

9 Detailed UI reference

9.1 Pulse-sequence controls

`Pulse shape` selects the analytic square branch or numerical Gaussian branch. `Bragg order n` accepts positive integers only. The two pulse-width fields accept positive finite values in microseconds. The first and third $\pi/2$ pulses always share the same width. The `Interrogation time T` field remains in milliseconds in both modes, but its physical definition changes as described in [section 3](#).

The first panel displays normalized intensity, not absolute optical power. Relative peak heights follow the area normalization. For example, $\tau_\pi = 2\tau_{\pi/2}$ gives equal square Rabi heights and, under the matched-envelope assumption, equal Gaussian peak Rabi rates. The FWHM arrows show the entered widths, and the blue arrows show T using the correct edge or centre convention.

9.2 Frequency controls

The minimum frequency must be strictly positive because the plot uses a logarithmic axis. The DC value is nevertheless evaluated in the self-tests and enforced exactly as $H_\phi(0) = 0$. The maximum frequency must

exceed the minimum. A very high maximum frequency combined with a broad Gaussian pulse may exceed the 4096-node quadrature safeguard; the UI then asks for a lower maximum frequency or a narrower pulse.

The point count controls only the requested frequency grid. It does not change the internal Gauss–Legendre order used inside Gaussian pulses. More points therefore improve displayed frequency resolution without changing the pulse-integration method.

9.3 Displayed status information

The status block reports the plotted sequence duration, peak effective Rabi rates, the active definition of T , the analytic or numerical integration branch, and the DC identity. In Gaussian mode the sequence duration includes the $\pm 6\sigma$ numerical windows. It is a plotting and integration extent, not a statement that a physical Gaussian field becomes exactly zero at those times.

9.4 Saving and inspecting figures

The Matplotlib toolbar provides zoom, pan, coordinate inspection and view reset. In the compressed pulse panel, the cursor readout converts the schematic coordinate back to relative physical time. The first pulse centre is the time origin. PDF or SVG export is preferred for reports because the line art remains vector-based; PNG is useful for slides or quick sharing.

10 Worked examples

10.1 Default first-order calculation

Set $n = 1$, $\tau_{\pi/2} = 25 \mu\text{s}$, $\tau_{\pi} = 50 \mu\text{s}$ and $T = 250 \text{ ms}$. In square mode, [equation \(8\)](#) gives

$$\frac{\Omega_{\pi/2}}{2\pi} = \frac{1}{4\tau_{\pi/2}} = 10 \text{ kHz}, \quad \frac{\Omega_{\pi}}{2\pi} = \frac{1}{2\tau_{\pi}} = 10 \text{ kHz}. \quad (34)$$

The transfer fringes repeat every 4 Hz. Finite-pulse filtering becomes prominent near the 10 kHz Rabi scale. In Gaussian mode, the same entered FWHM values and matched-intensity assumption give, from [equation \(18\)](#),

$$\frac{\Omega_{0,\pi/2}}{2\pi} = \frac{\Omega_{0,\pi}}{2\pi} \simeq 9.394 \text{ kHz}. \quad (35)$$

The low-frequency response remains nearly identical because the pulse areas and centre separation dominate there. The Gaussian high-frequency envelope decreases more rapidly.

10.2 Changing Bragg order

Changing n from 1 to 3 leaves $s(t)$ unchanged. The plotted magnitude triples and the PSD transfer factor increases by nine:

$$|H_{\varphi}^{(3)}| = 3|H_{\varphi}^{(1)}|, \quad |H_{\varphi}^{(3)}|^2 = 9|H_{\varphi}^{(1)}|^2. \quad (36)$$

This scaling is a phase-gain convention, not a full prediction of third-order Bragg diffraction efficiency.

10.3 Evaluating a modulation line

Suppose a differential frequency modulation has peak deviation $\Delta\nu = 10 \text{ Hz}$ at $f_m = 100 \text{ Hz}$. Its phase-modulation peak is $\Delta\nu/f_m = 0.1 \text{ rad}$. If the UI gives $|H_{\varphi}(f_m)| = 2.5$, the atom-phase peak is 0.25 rad, and the integrated one-sided PSD line power is $0.25^2/2 = 0.03125 \text{ rad}^2$.

11 Validation and reproducibility

Run the built-in test suite without opening the GUI:

```
python "bragg_transfer_function_ui.py" --self-test
```

Table 2. Automated checks implemented by the application.

Check	Purpose
Exact DC response	Enforces $H_\varphi(0) = 0$ for a closed interferometer sequence.
Sensitivity boundaries	Confirms $0 \rightarrow +1 \rightarrow -1 \rightarrow 0$ at all pulse boundaries.
Square analytic/numerical comparison	Compares the stable analytic expression with a high-resolution trapezoidal reference.
Rabi-frequency regularity	Confirms finite values at removable singularities $\omega = \pm\Omega$.
Pulse-area changes	Confirms Gaussian derivative integrals $+1, -2, +1$.
Bragg-order scaling	Confirms linear scaling of complex H_φ with n .
Instantaneous limit	Confirms convergence toward $4n \sin^2(\omega T/2)$ as pulse widths vanish.
GUI smoke tests	Confirms that both pulse modes construct and plot without callback errors.

The manual figures can be regenerated with

```
python "manual_figures.py"
```

This command writes vector PDF files into `figures/`. It uses the same calculation functions as the GUI, ensuring that equations, examples and plotted results remain synchronized with the application.

✓ Required physical identities

For every valid parameter set, verify $s(-\infty) = s(+\infty) = 0$, $H_\varphi(0) = 0$, linear n scaling and convergence to the instantaneous-pulse result when the widths become negligible compared with T .

12 Troubleshooting

Symptom	Explanation and remedy
The high-frequency curve looks random.	The logarithmic grid undersamples fringes spaced by approximately $1/T$. Narrow the frequency interval and increase the point count, or use a locally averaged PSD factor.
Square and Gaussian curves look almost identical.	This is expected below the inverse pulse duration. Compare their high-frequency envelopes or use wider pulses while keeping the pulse areas normalized.
Gaussian calculation reports overlapping pulses.	The $\pm 6\sigma$ numerical windows overlap. Increase centre spacing T or use a full overlapping-pulse Hamiltonian, which is outside this application.
Gaussian calculation requests too many quadrature nodes.	Reduce $f_{\max}$ or use a narrower pulse. The 4096-node limit prevents excessive runtime and memory use.
The sensitivity transitions are invisible.	The full sequence time is much longer than the pulse widths. Use the compressed pulse panel to inspect FWHM and shape; zoom around a pulse in the sensitivity panel if needed.

Symptom	Explanation and remedy
Changing n does not alter $s(t)$.	This is intentional. The MIGA factor n multiplies the phase transfer function outside the normalized sensitivity function.
The inferred frequency-noise response is too large or too small.	Confirm that the input is the differential phase at the atoms. Common-mode rejection, optical delay and AOM topology are not included automatically.

13 Known limitations and extension path

The current software is deliberately a reduced model. It is reliable for ideal resonant, separated pulses that can be described by a two-level rotation with a known effective Rabi envelope. The following effects require a more complete treatment:

- detuning and Doppler-dependent velocity distributions;
- simultaneous coupling of multiple momentum states in higher-order Bragg diffraction;
- non-ideal diffraction efficiency and parasitic interferometers;
- AC Stark shifts, spontaneous emission and laser-intensity noise;
- optical-cavity build-up, cavity pole filtering and propagation delay;
- overlapping pulses or arbitrary chirped/phase-shaped envelopes;
- correlations between the two optical fields and spatial phase variations across the atomic cloud.

The natural extension is to propagate the full time-dependent Hamiltonian, compute the interferometer output phase under a small phase perturbation, and obtain the sensitivity function by functional differentiation. The present UI, plotting and PSD layers can then be retained while replacing the pulse-dynamics backend.

14 Code organization

The application is intentionally distributed as a single executable Python file. `InterferometerParameters` validates SI-valued parameters and defines the square schedule. `sensitivity_function` and `transfer_function` implement the square branch. `gaussian_sensitivity_function` and `gaussian_transfer_function` implement the Gaussian branch. `BraggTransferFunctionApp` owns the Tkinter controls and Matplotlib canvas. The `run_self_test` function provides a dependency-free validation entry point.

All physical calculations use seconds, hertz and radians internally. Unit conversion occurs only when values are read from the UI or reported to the user. This separation prevents microsecond/millisecond conversion errors from propagating into the model.

15 Using this manual on Overleaf

Upload `bragg_transfer_function_manual.tex` together with the `figures/` directory. Set the main document to the manual file and compile with pdfLaTeX. The document uses packages included in current TeX Live and Overleaf distributions, including `newtx`, `fontawesome5`, `tcolorbox`, `tikz`, `siunitx` and `cleveref`. No shell escape, external bibliography processor or custom font upload is required.

The source is a technical manual with Nature-inspired typography, accessible prose, numbered references, double-column-width vector figures and colorblind-safe colors. It is not an official Nature template and should not be submitted as a journal manuscript without adapting it to the current instructions of the target Nature Portfolio journal.

Code availability

The calculator, manual source and figure-generation script are maintained in the MIGA programme repository at

github.com/Quantum00man/MIGA-program/tree/main/bragg-transfer-function.

References

- [1] Canuel, B. *et al.* Exploring gravity with the MIGA large scale atom interferometer. *Sci. Rep.* **8**, 14064 (2018). doi:10.1038/s41598-018-32165-z.
- [2] Cheinet, P., Canuel, B., Pereira Dos Santos, F., Gauguier, A., Yver-Leduc, F. & Landragin, A. Measurement of the sensitivity function in a time-domain atomic interferometer. *IEEE Trans. Instrum. Meas.* **57**, 1141–1148 (2008). doi:10.1109/TIM.2007.915148.